

Autoencoders

Weekend 2, Friday morning

Carlos Cotrini

Friday 18 September 2026

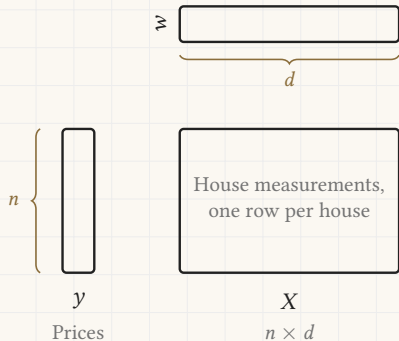
1

Section 1

Machine learning needs vectors

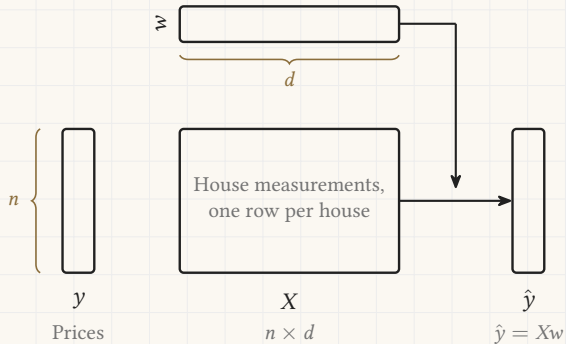
Linear regression with gradient descent

Two weeks ago, in one picture. Each box is a piece of data, and its shape is the shape of the data.



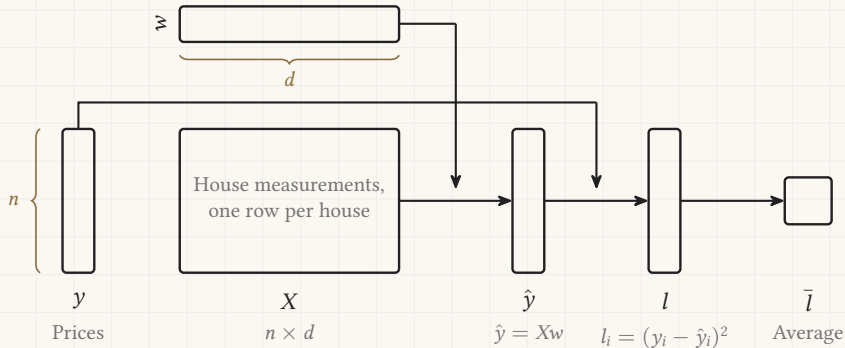
Linear regression with gradient descent

Two weeks ago, in one picture. Each box is a piece of data, and its shape is the shape of the data.



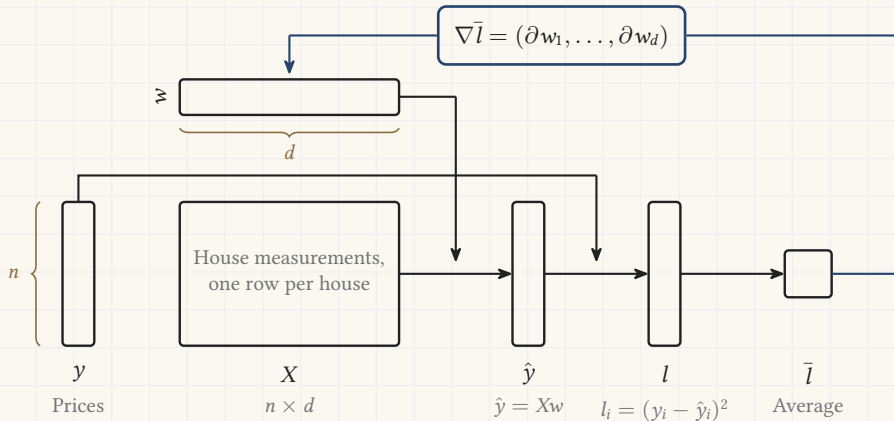
Linear regression with gradient descent

Two weeks ago, in one picture. Each box is a piece of data, and its shape is the shape of the data.

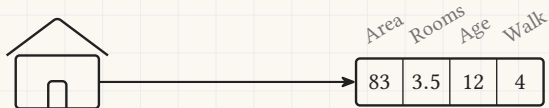


Linear regression with gradient descent

Two weeks ago, in one picture. Each box is a piece of data, and its shape is the shape of the data.

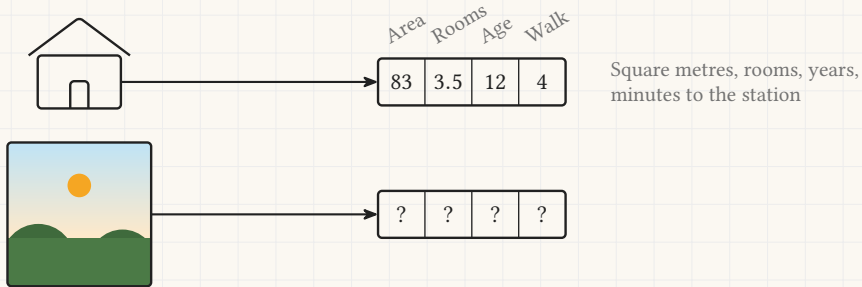


How do we do machine learning with images and text?

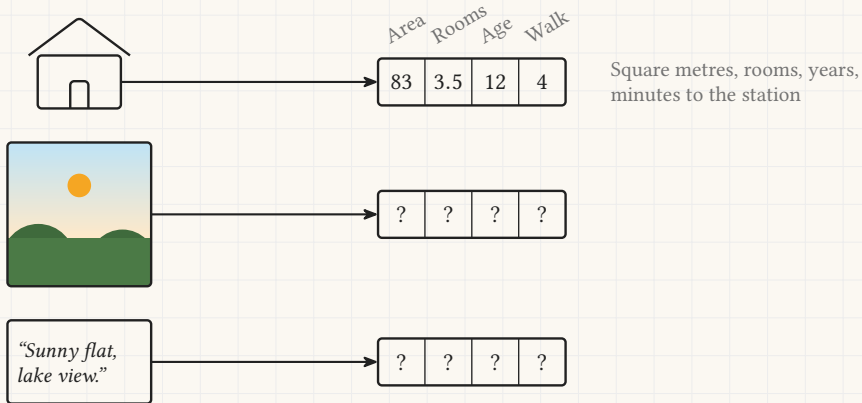


Square metres, rooms, years,
minutes to the station

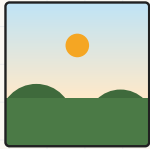
How do we do machine learning with images and text?



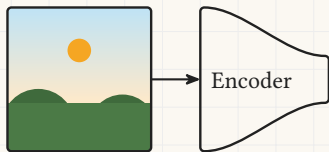
How do we do machine learning with images and text?



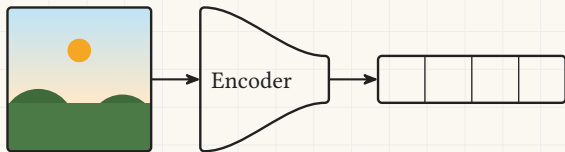
Autoencoders



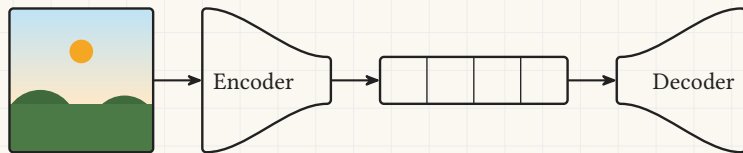
Autoencoders



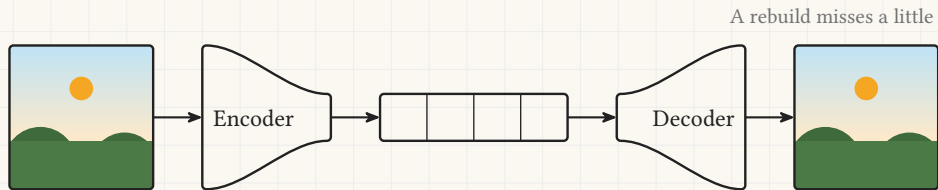
Autoencoders



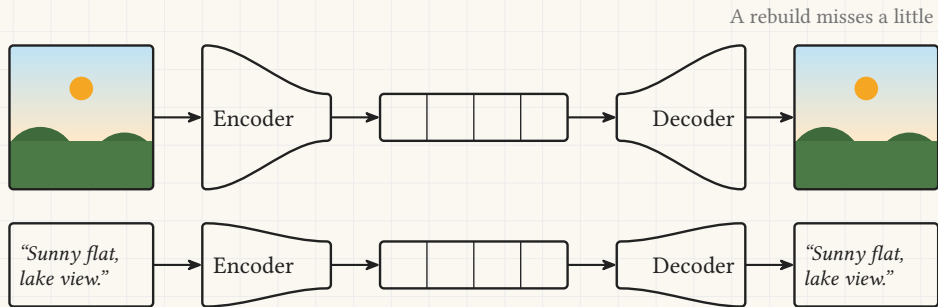
Autoencoders



Autoencoders



Autoencoders



2

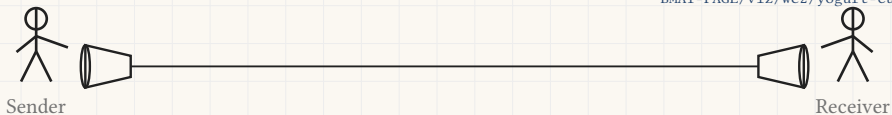
Section 2

The yogurt cup game

The yogurt cup game



[https://eth-bmai-hs26.github.io/
BMAI-PAGE/viz/we2/yogurt-cups/](https://eth-bmai-hs26.github.io/BMAI-PAGE/viz/we2/yogurt-cups/)

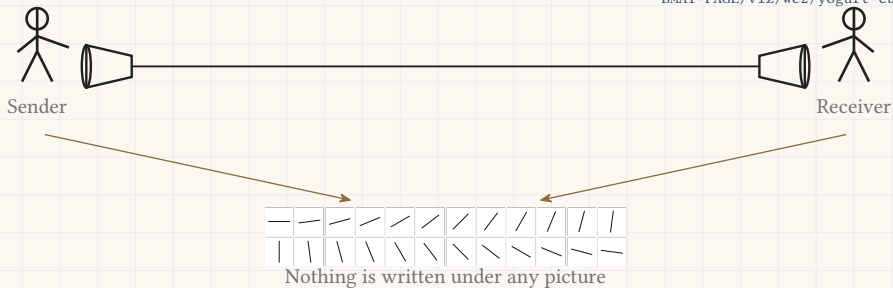


The yogurt cup game

- Both see every picture



<https://eth-bmai-hs26.github.io/BMAI-PAGE/viz/we2/yogurt-cups/>

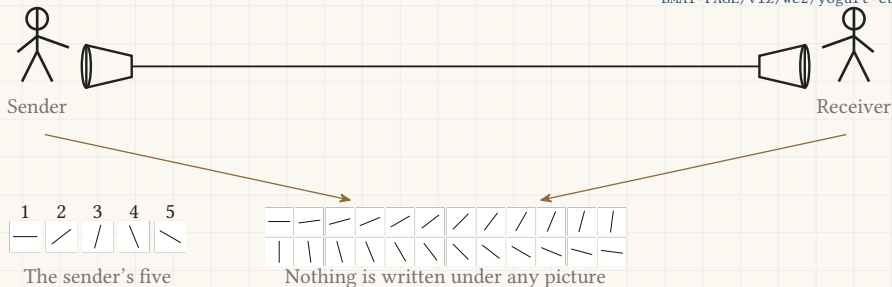


The yogurt cup game

- Both see every picture
- One sees five of them, in order



<https://eth-bmai-hs26.github.io/BMAI-PAGE/viz/we2/yogurt-cups/>

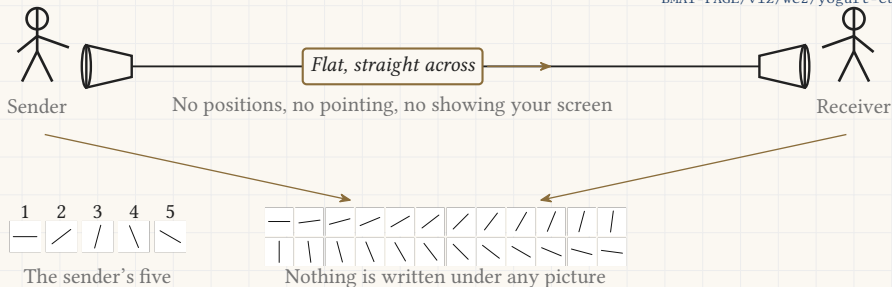


The yogurt cup game

- Both see every picture
- One sees five of them, in order
- Only words go through the cup



<https://eth-bmai-hs26.github.io/BMAI-PAGE/viz/we2/yogurt-cups/>

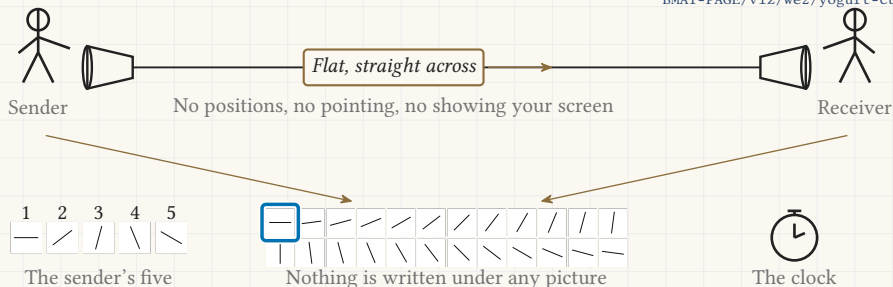


The yogurt cup game

- Both see every picture
- One sees five of them, in order
- Only words go through the cup
- The partner finds them, against the clock



<https://eth-bmai-hs26.github.io/BMAI-PAGE/viz/we2/yogurt-cups/>



The score is the time

Round 1

Steep, nearly upright



The clock

The score is the time

Round 1

Steep, nearly upright



"Not that one. The clock is still running."



The clock

The score is the time

Round 1

Steep, nearly upright



"Not that one. The clock is still running."



The clock

The score is the time

Round 1

Steep, nearly upright



"Not that one. The clock is still running."



The clock

Between rounds

*"The pairs who get fast are the ones
who stop describing and start counting."*

The score is the time

Round 1

Steep, nearly upright



"Not that one. The clock is still running."



The clock

Between rounds

*"The pairs who get fast are the ones
who stop describing and start counting."*

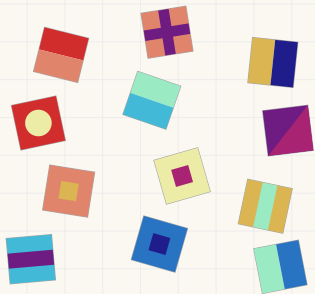
Round 2

Ten steps up from flat



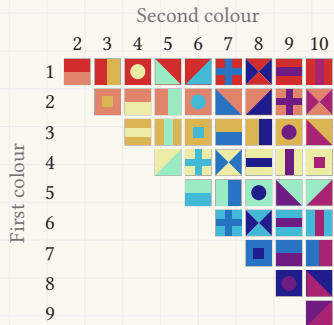
Right, first time

Words are a coarse code



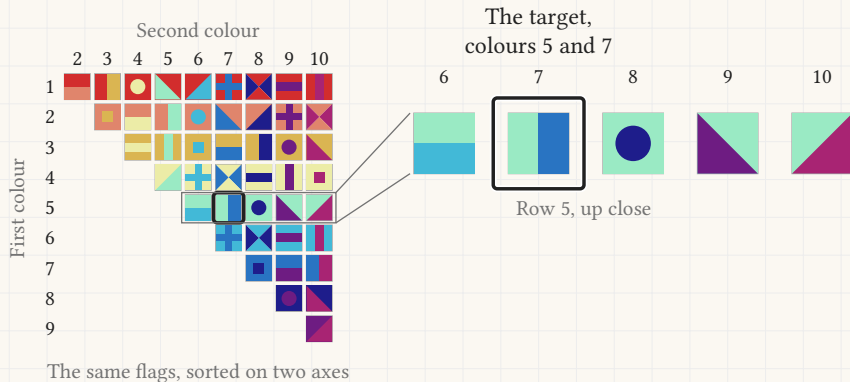
Twelve of them, in no order

Words are a coarse code

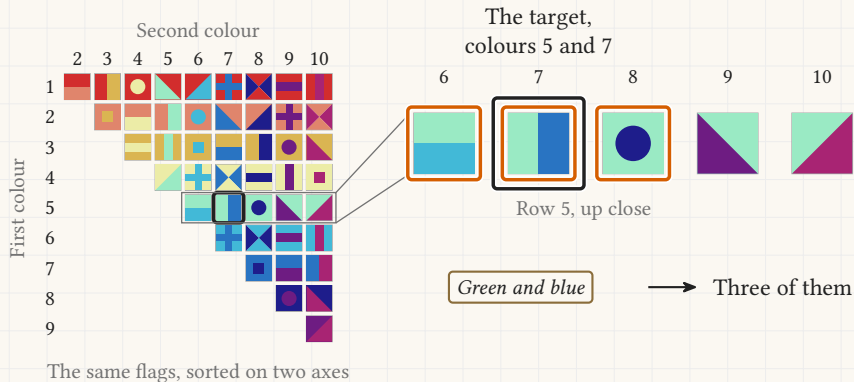


The same flags, sorted on two axes

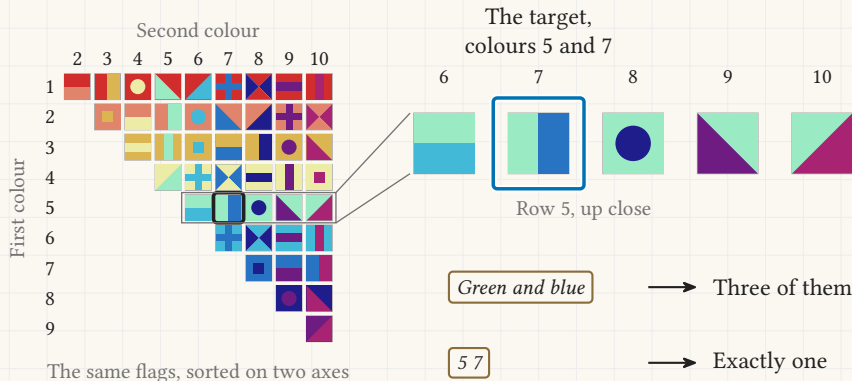
Words are a coarse code



Words are a coarse code

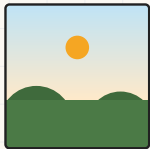


Words are a coarse code



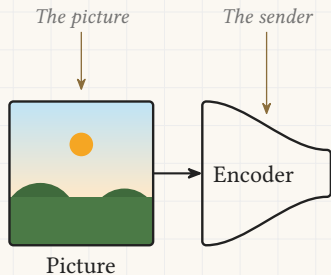
The game, named

The picture

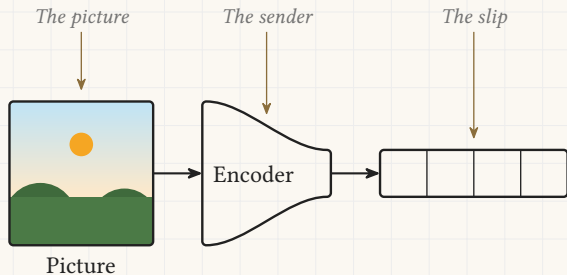


Picture

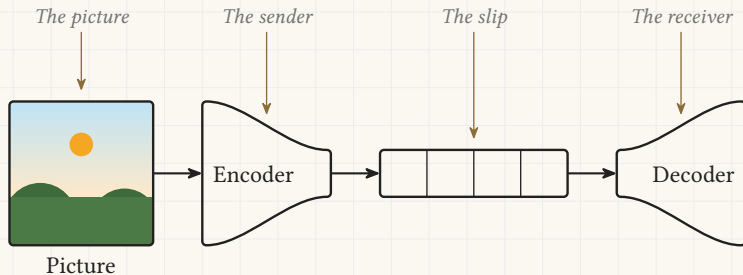
The game, named



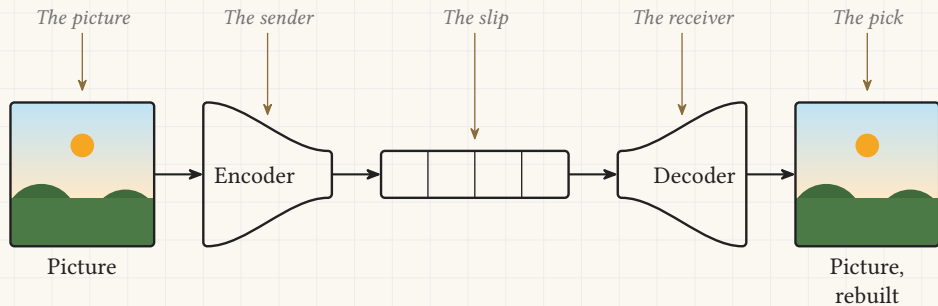
The game, named



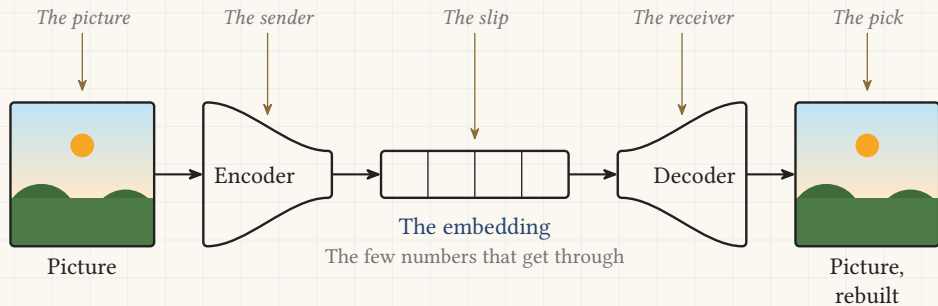
The game, named



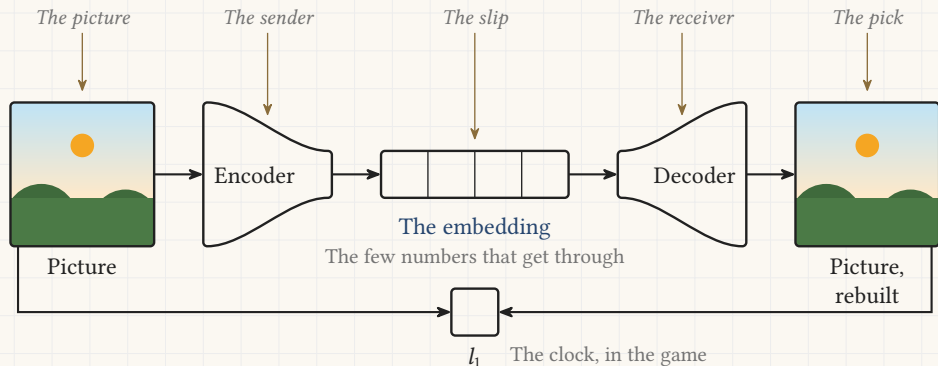
The game, named



The game, named



The game, named



3

Section 3

An autoencoder for images

An autoencoder for images

So how do we do this with images?

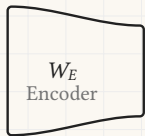


Pictures

64×64 pixels

An autoencoder for images

So how do we do this with images?

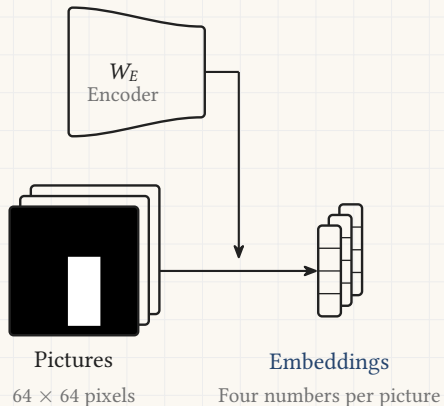


Pictures

64×64 pixels

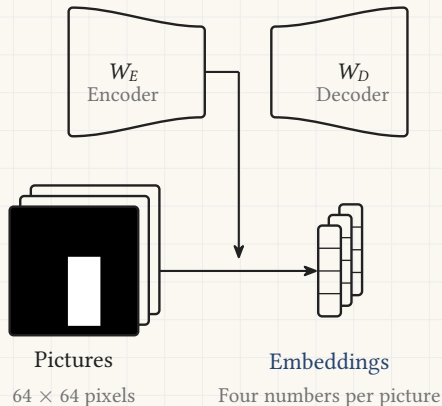
An autoencoder for images

So how do we do this with images?



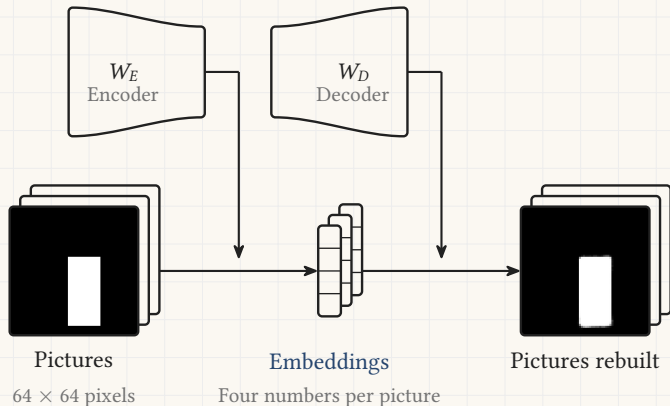
An autoencoder for images

So how do we do this with images?



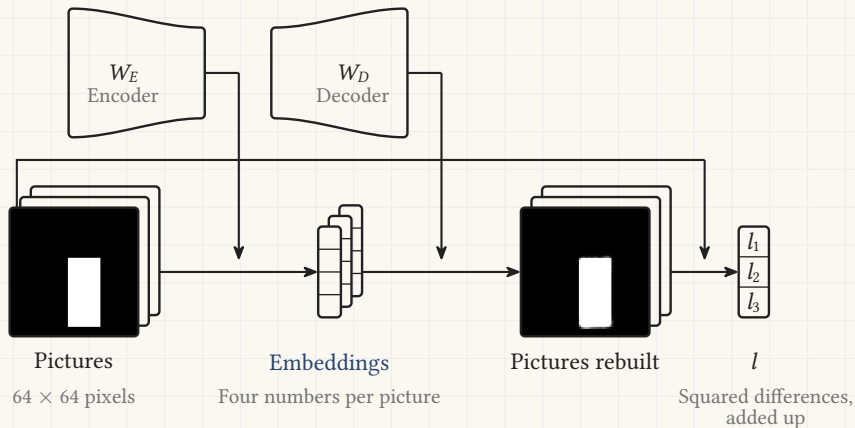
An autoencoder for images

So how do we do this with images?



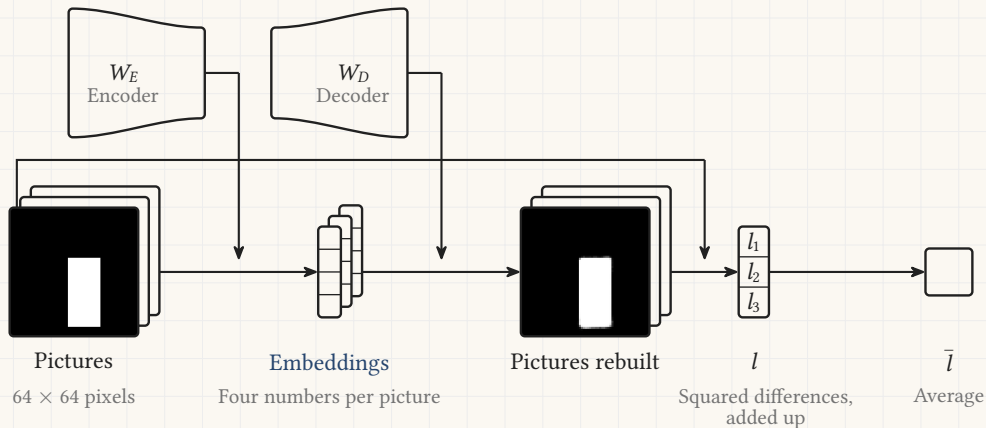
An autoencoder for images

So how do we do this with images?



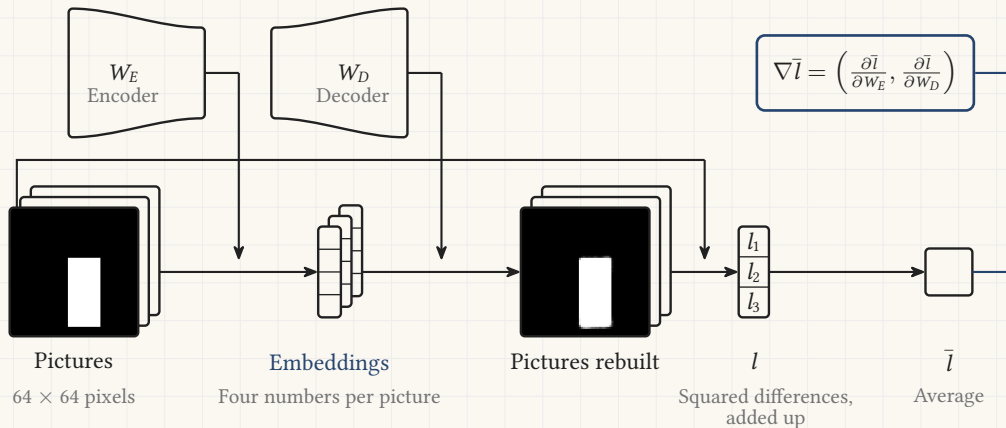
An autoencoder for images

So how do we do this with images?



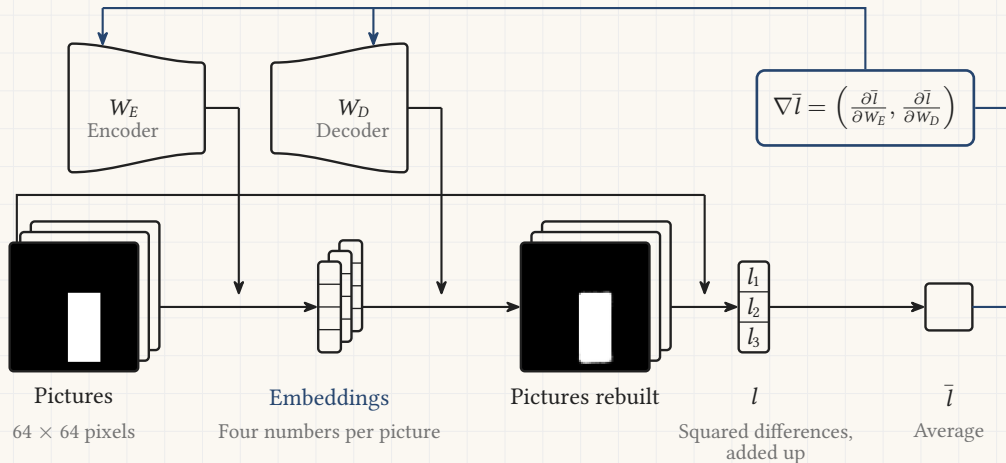
An autoencoder for images

So how do we do this with images?

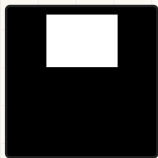


An autoencoder for images

So how do we do this with images?



The loss of one picture

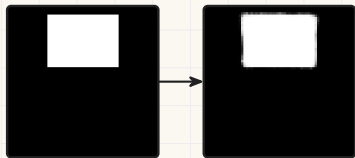


The picture

64 × 64 pixels

The third picture

The loss of one picture



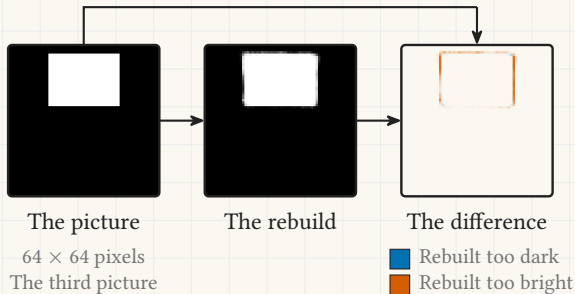
The picture

64×64 pixels

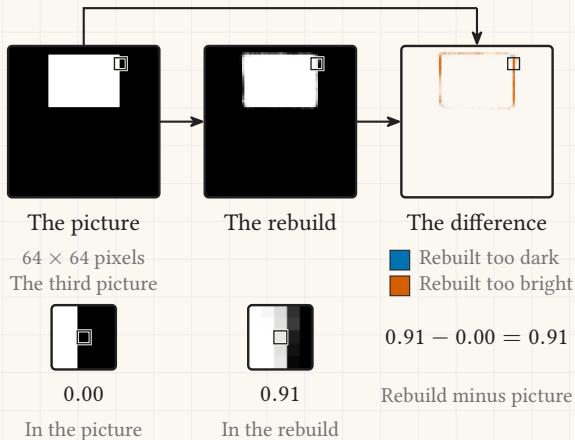
The third picture

The rebuild

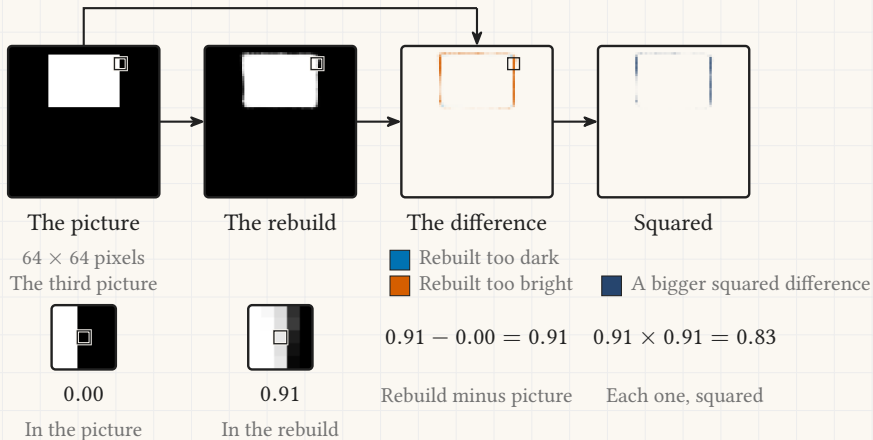
The loss of one picture



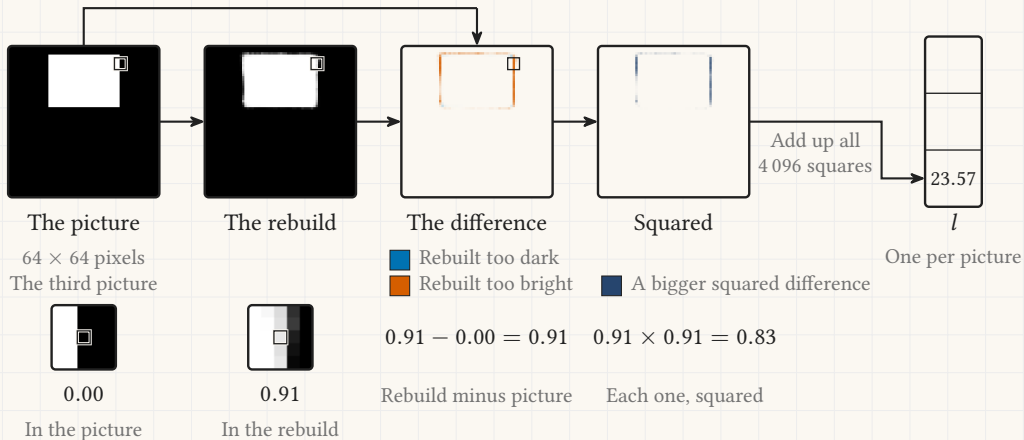
The loss of one picture



The loss of one picture



The loss of one picture



4

Section 4

An autoencoder in PyTorch

The data

One seeded run. Your own run will differ.

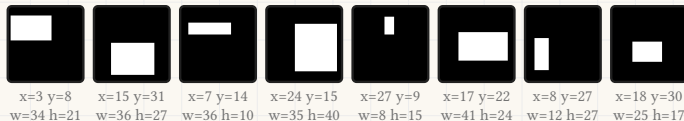


https://colab.research.google.com/github/eth-bmai-hs26/w2-cx-public/blob/main/autoencoders-basic/autoencoders_basic.ipynb

The notebook

The data

One seeded run. Your own run will differ.



Four numbers, one picture

64 × 64 pixels, black,
one white rectangle

A family of pictures, like the flags

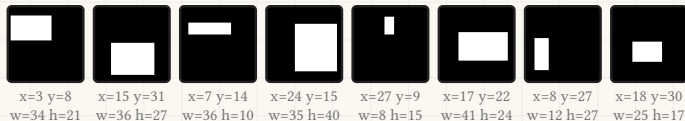


https://colab.research.google.com/github/eth-bmai-hs26/w2-cx-public/blob/main/autoencoders-basic/autoencoders_basic.ipynb

The notebook

The data

One seeded run. Your own run will differ.



Four numbers, one picture

64 × 64 pixels, black,
one white rectangle

A family of pictures, like the flags

```
IMG_SIZE = 64
```

```
x = np.random.randint(2, IMG_SIZE // 2)
y = np.random.randint(2, IMG_SIZE // 2)
w = np.random.randint(8, IMG_SIZE - x - 2)
h = np.random.randint(8, IMG_SIZE - y - 2)
```

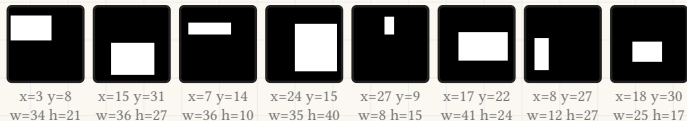


https://colab.research.google.com/github/eth-bmai-hs26/w2-cx-public/blob/main/autoencoders-basic/autoencoders_basic.ipynb

The notebook

The data

One seeded run. Your own run will differ.



Four numbers, one picture

64 × 64 pixels, black,
one white rectangle

A family of pictures, like the flags

```
IMG_SIZE = 64
```

```
x = np.random.randint(2, IMG_SIZE // 2)
y = np.random.randint(2, IMG_SIZE // 2)
w = np.random.randint(8, IMG_SIZE - x - 2)
h = np.random.randint(8, IMG_SIZE - y - 2)
```

```
img[0, y:y+h, x:x+w] = 1.0
```

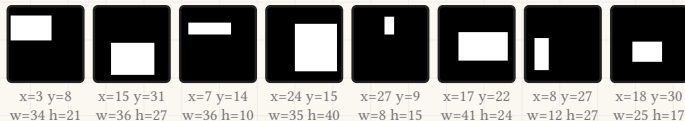


https://colab.research.google.com/github/eth-bmai-hs26/w2-cx-public/blob/main/autoencoders-basic/autoencoders_basic.ipynb

The notebook

The data

One seeded run. Your own run will differ.



Four numbers, one picture

64 × 64 pixels, black,
one white rectangle

A family of pictures, like the flags

```
IMG_SIZE = 64
```

```
x = np.random.randint(2, IMG_SIZE // 2)
y = np.random.randint(2, IMG_SIZE // 2)
w = np.random.randint(8, IMG_SIZE - x - 2)
h = np.random.randint(8, IMG_SIZE - y - 2)
```

```
img[0, y:y+h, x:x+w] = 1.0
```

```
train_data = RectangleDataset(8000)
test_data = RectangleDataset(500)
```

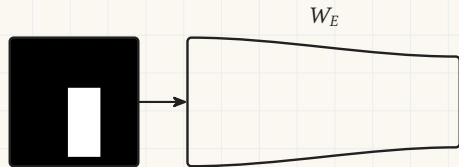


https://colab.research.google.com/github/eth-bmai-hs26/w2-cx-public/blob/main/autoencoders-basic/autoencoders_basic.ipynb

The notebook

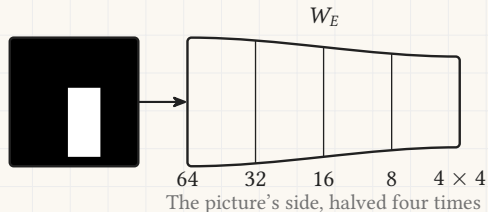
8 000 pictures to train on, 500 held back

The encoder



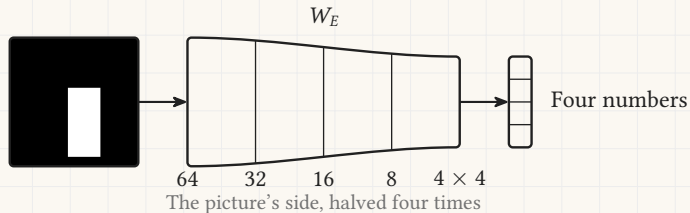
```
class Encoder(nn.Module):  
    def __init__(self):  
        super().__init__()
```

The encoder



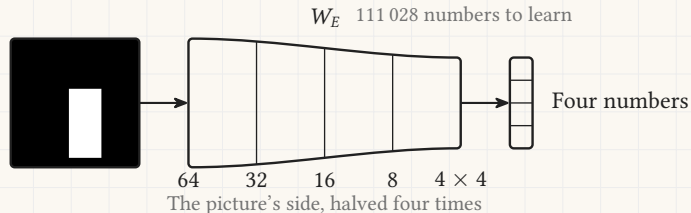
```
class Encoder(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 16, kernel_size=4, stride=2, padding=1)  
        self.conv2 = nn.Conv2d(16, 32, kernel_size=4, stride=2, padding=1)  
        self.conv3 = nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1)  
        self.conv4 = nn.Conv2d(64, 64, kernel_size=4, stride=2, padding=1)
```

The encoder



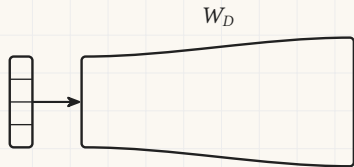
```
class Encoder(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 16, kernel_size=4, stride=2, padding=1)  
        self.conv2 = nn.Conv2d(16, 32, kernel_size=4, stride=2, padding=1)  
        self.conv3 = nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1)  
        self.conv4 = nn.Conv2d(64, 64, kernel_size=4, stride=2, padding=1)  
        self.fc = nn.Linear(64 * 4 * 4, LATENT_DIM)  
        self.relu = nn.ReLU()
```

The encoder



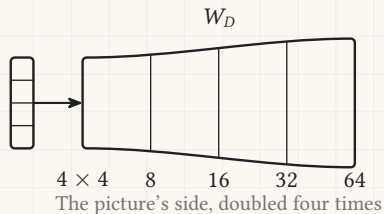
```
class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=4, stride=2, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=4, stride=2, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1)
        self.conv4 = nn.Conv2d(64, 64, kernel_size=4, stride=2, padding=1)
        self.fc = nn.Linear(64 * 4 * 4, LATENT_DIM)
        self.relu = nn.ReLU()
```


The decoder



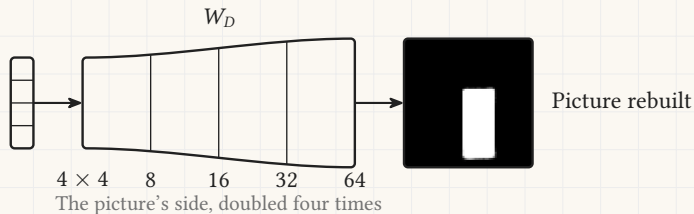
```
class Decoder(nn.Module):  
    def __init__(self):  
        super().__init__()
```

The decoder



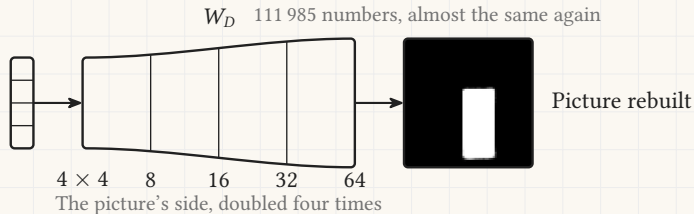
```
class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(LATENT_DIM, 64 * 4 * 4)
        self.deconv1 = nn.ConvTranspose2d(64, 64, kernel_size=4, stride=2, padding=1)
        self.deconv2 = nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1)
        self.deconv3 = nn.ConvTranspose2d(32, 16, kernel_size=4, stride=2, padding=1)
        self.deconv4 = nn.ConvTranspose2d(16, 1, kernel_size=4, stride=2, padding=1)
```

The decoder



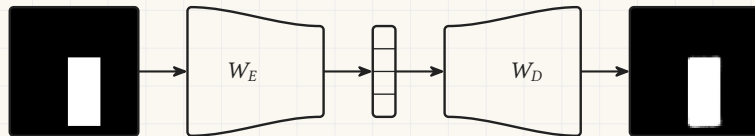
```
class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(LATENT_DIM, 64 * 4 * 4)
        self.deconv1 = nn.ConvTranspose2d(64, 64, kernel_size=4, stride=2, padding=1)
        self.deconv2 = nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1)
        self.deconv3 = nn.ConvTranspose2d(32, 16, kernel_size=4, stride=2, padding=1)
        self.deconv4 = nn.ConvTranspose2d(16, 1, kernel_size=4, stride=2, padding=1)
        self.relu = nn.ReLU()
```

The decoder

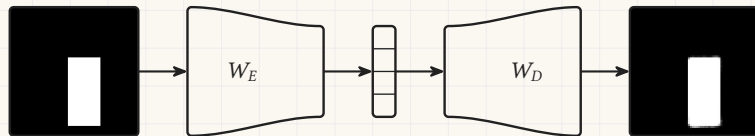


```
class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(LATENT_DIM, 64 * 4 * 4)
        self.deconv1 = nn.ConvTranspose2d(64, 64, kernel_size=4, stride=2, padding=1)
        self.deconv2 = nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2, padding=1)
        self.deconv3 = nn.ConvTranspose2d(32, 16, kernel_size=4, stride=2, padding=1)
        self.deconv4 = nn.ConvTranspose2d(16, 1, kernel_size=4, stride=2, padding=1)
        self.relu = nn.ReLU()
```

The two halves, one model

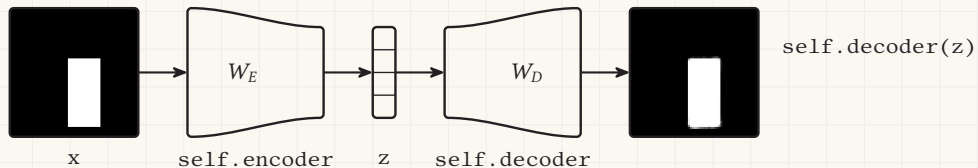


The two halves, one model



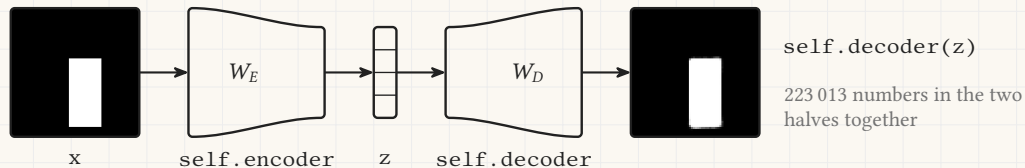
```
class Autoencoder(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.encoder = Encoder()  
        self.decoder = Decoder()
```

The two halves, one model



```
class Autoencoder(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.encoder = Encoder()  
        self.decoder = Decoder()  
  
    def forward(self, x):  
        z = self.encoder(x)  
        return self.decoder(z), z
```

The two halves, one model

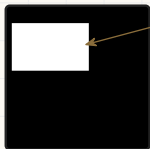


```
class Autoencoder(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.encoder = Encoder()  
        self.decoder = Decoder()  
  
    def forward(self, x):  
        z = self.encoder(x)  
        return self.decoder(z), z
```

```
device = 'cuda' if torch.cuda.is_available() else 'cpu'  
model = Autoencoder().to(device)
```


The optimizer and the loss

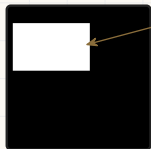
One seeded run. Your own run will differ.



White pixels are 16.8 percent of the training set. Most of every picture is black.

The optimizer and the loss

One seeded run. Your own run will differ.

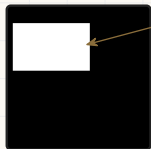


White pixels are 16.8 percent of the training set. Most of every picture is black.

```
optimizer = optim.Adam(model.parameters(), lr=3e-3)
```

The optimizer and the loss

One seeded run. Your own run will differ.



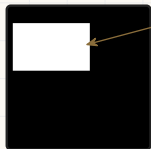
White pixels are 16.8 percent of the training set. Most of every picture is black.

```
optimizer = optim.Adam(model.parameters(), lr=3e-3)
```

```
loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([5.0]).to(device))
```

The optimizer and the loss

One seeded run. Your own run will differ.



White pixels are 16.8 percent of the training set. Most of every picture is black.



A white pixel

Counted five times
when it is wrong



A black pixel

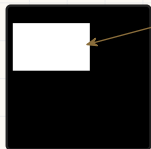
Counted once
when it is wrong

```
optimizer = optim.Adam(model.parameters(), lr=3e-3)
```

```
loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([5.0]).to(device))
```

The optimizer and the loss

One seeded run. Your own run will differ.



White pixels are 16.8 percent of the training set. Most of every picture is black.



A white pixel

Counted five times
when it is wrong



A black pixel

Counted once
when it is wrong

The same job as l , the squared differences added up: one number saying how wrong the rebuilt picture is.

```
optimizer = optim.Adam(model.parameters(), lr=3e-3)
```

```
loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([5.0]).to(device))
```

The training loop

```
for epoch in range(20):  
    model.train()  
    total_loss = 0  
    for imgs, _ in train_loader:  
        imgs = imgs.to(device)
```

In the picture In the notebook

The training loop

```
for epoch in range(20):  
    model.train()  
    total_loss = 0  
    for imgs, _ in train_loader:  
        imgs = imgs.to(device)  
        optimizer.zero_grad()
```

In the picture

The old gradient, cleared

In the notebook

`optimizer.zero_grad()`

The training loop

```
for epoch in range(20):  
    model.train()  
    total_loss = 0  
    for imgs, _ in train_loader:  
        imgs = imgs.to(device)  
        optimizer.zero_grad()  
        logits, _ = model(imgs)
```

In the picture

The old gradient, cleared
Through W_E and W_D

In the notebook

`optimizer.zero_grad()`
`model(imgs)`

The training loop

```
for epoch in range(20):  
    model.train()  
    total_loss = 0  
    for imgs, _ in train_loader:  
        imgs = imgs.to(device)  
        optimizer.zero_grad()  
        logits, _ = model(imgs)  
        loss = loss_fn(logits, imgs)
```

In the picture

The old gradient, cleared
Through W_E and W_D
The losses, and $\bar{l}$

In the notebook

```
optimizer.zero_grad()  
model(imgs)  
loss_fn(logits, imgs)
```

The training loop

```
for epoch in range(20):  
    model.train()  
    total_loss = 0  
    for imgs, _ in train_loader:  
        imgs = imgs.to(device)  
        optimizer.zero_grad()  
        logits, _ = model(imgs)  
        loss = loss_fn(logits, imgs)  
        loss.backward()
```

In the picture

The old gradient, cleared
Through W_E and W_D
The losses, and $\bar{l}$
 $\nabla \bar{l}$, both halves at once

In the notebook

```
optimizer.zero_grad()  
model(imgs)  
loss_fn(logits, imgs)  
loss.backward()
```

The training loop

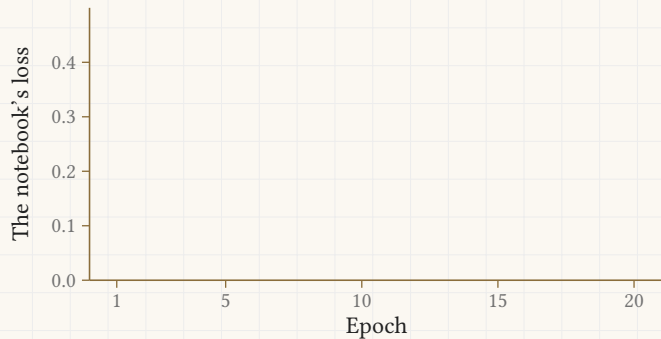
```
for epoch in range(20):  
    model.train()  
    total_loss = 0  
    for imgs, _ in train_loader:  
        imgs = imgs.to(device)  
        optimizer.zero_grad()  
        logits, _ = model(imgs)  
        loss = loss_fn(logits, imgs)  
        loss.backward()  
        optimizer.step()  
        total_loss += loss.item()
```

In the picture
The old gradient, cleared
Through W_E and W_D
The losses, and $\bar{l}$
 $\nabla \bar{l}$, both halves at once
 W_E and W_D move

In the notebook
`optimizer.zero_grad()`
`model(imgs)`
`loss_fn(logits, imgs)`
`loss.backward()`
`optimizer.step()`

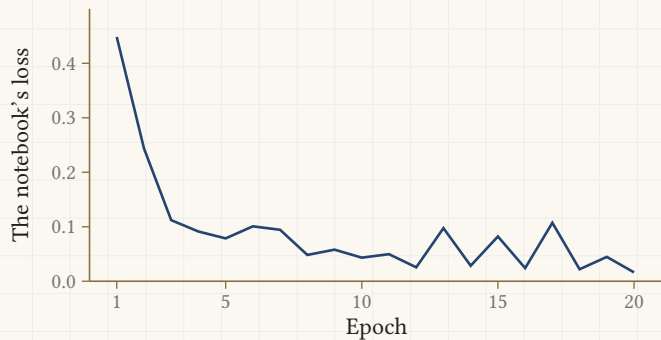
The loss curve

One seeded run. Your own run will differ.



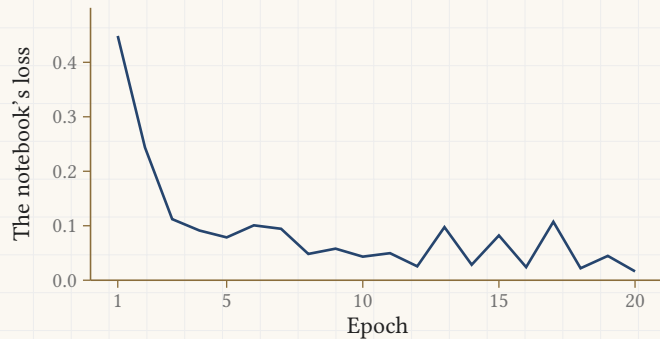
The loss curve

One seeded run. Your own run will differ.



The loss curve

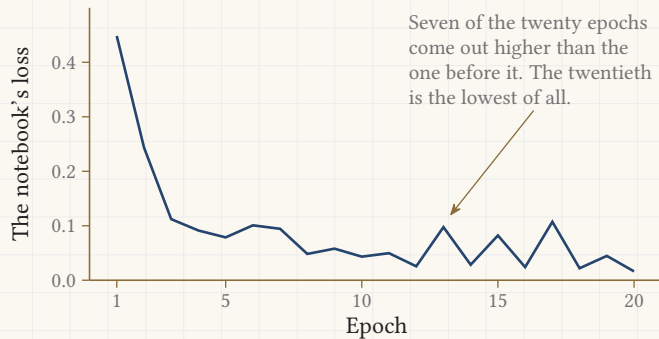
One seeded run. Your own run will differ.



Epoch	1	loss=0.44838
Epoch	2	loss=0.24371
Epoch	3	loss=0.11211
	...	
Epoch	18	loss=0.02196
Epoch	19	loss=0.04468
Epoch	20	loss=0.01623

The loss curve

One seeded run. Your own run will differ.



Epoch	1	loss=0.44838
Epoch	2	loss=0.24371
Epoch	3	loss=0.11211
	...	
Epoch	18	loss=0.02196
Epoch	19	loss=0.04468
Epoch	20	loss=0.01623

Pictures in, pictures out

One seeded run. Your own run will differ.

Given



```
model.eval()  
imgs, _ = next(iter(test_loader))  
with torch.no_grad():
```


Pictures in, pictures out

One seeded run. Your own run will differ.

Given



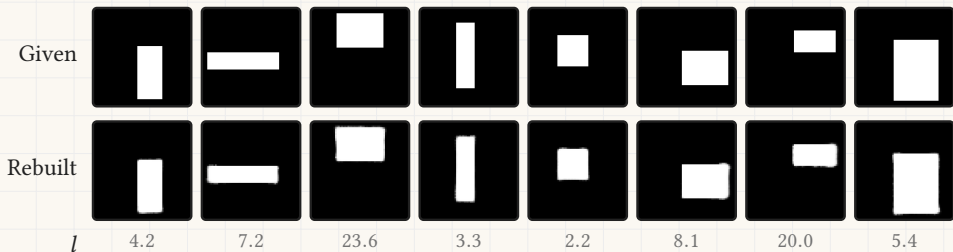
Rebuilt



```
model.eval()  
imgs, _ = next(iter(test_loader))  
with torch.no_grad():  
    logits, _ = model(imgs.to(device))  
    recons = torch.sigmoid(logits).cpu()
```

Pictures in, pictures out

One seeded run. Your own run will differ.

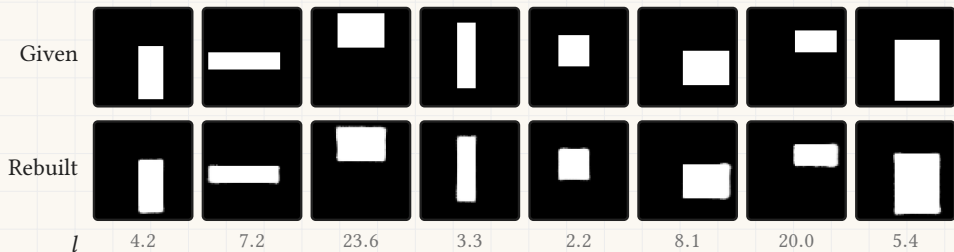


```
model.eval()  
imgs, _ = next(iter(test_loader))  
with torch.no_grad():  
    logits, _ = model(imgs.to(device))  
    recons = torch.sigmoid(logits).cpu()
```

l is the squared loss, added up over the pixels. Its scale is unrelated to the epoch numbers.

Pictures in, pictures out

One seeded run. Your own run will differ.



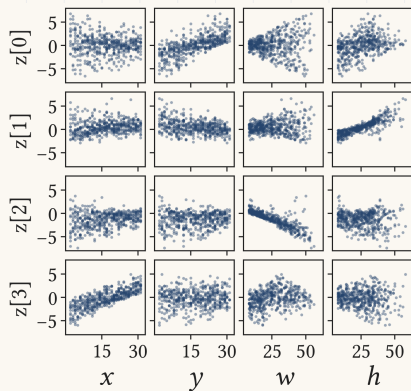
```
model.eval()  
imgs, _ = next(iter(test_loader))  
with torch.no_grad():  
    logits, _ = model(imgs.to(device))  
    recons = torch.sigmoid(logits).cpu()
```

l is the squared loss, added up over the pixels. Its scale is unrelated to the epoch numbers.

Over the whole test set, 96 percent of the miss lies within two pixels of the rectangle's edge.

What the four numbers learned

One seeded run. Your own run will differ.



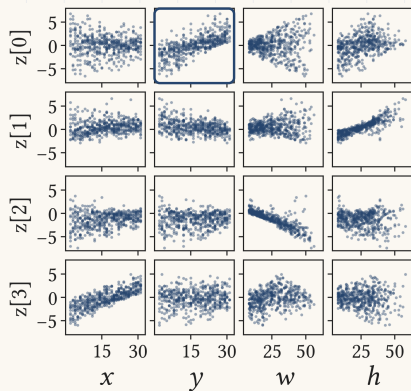
```
_, z = model(imgs.to(device))
```

```
param_names = ['x', 'y', 'w', 'h']
```

Each number against the four it was never given

What the four numbers learned

One seeded run. Your own run will differ.



```
_, z = model(imgs.to(device))
```

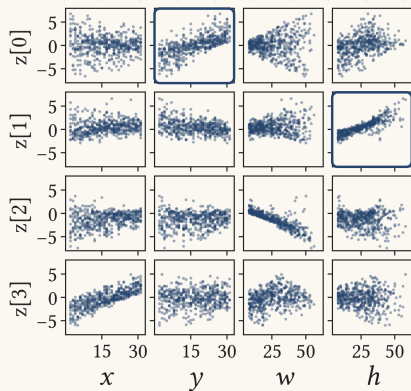
```
param_names = ['x', 'y', 'w', 'h']
```

Each number against the four it was never given

$z[0]$ rises with y , the top edge +0.69

What the four numbers learned

One seeded run. Your own run will differ.



```
_, z = model(imgs.to(device))
```

```
param_names = ['x', 'y', 'w', 'h']
```

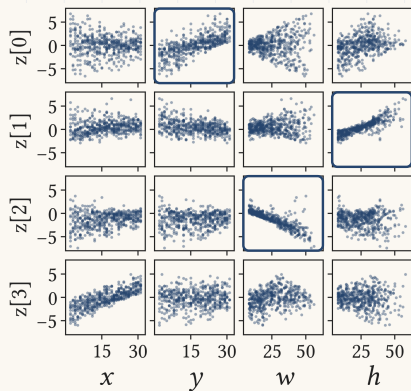
Each number against the four it was never given

$z[0]$ rises with y , the top edge +0.69

$z[1]$ rises with h , the height +0.85

What the four numbers learned

One seeded run. Your own run will differ.



```
_, z = model(imgs.to(device))
```

```
param_names = ['x', 'y', 'w', 'h']
```

Each number against the four it was never given

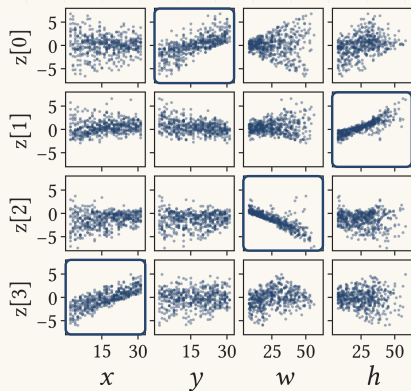
$z[0]$ rises with y , the top edge +0.69

$z[1]$ rises with h , the height +0.85

$z[2]$ falls as w , the width, grows -0.83

What the four numbers learned

One seeded run. Your own run will differ.



```
_, z = model(imgs.to(device))
```

```
param_names = ['x', 'y', 'w', 'h']
```

Each number against the four it was never given

$z[0]$ rises with y , the top edge +0.69

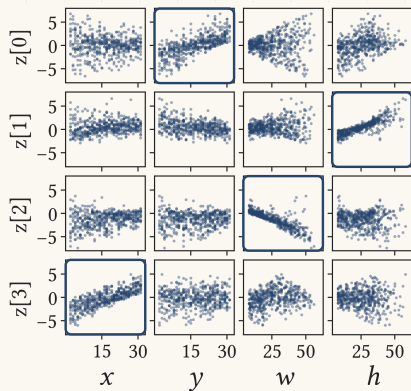
$z[1]$ rises with h , the height +0.85

$z[2]$ falls as w , the width, grows -0.83

$z[3]$ rises with x , the left edge +0.78

What the four numbers learned

One seeded run. Your own run will differ.



```
_, z = model(imgs.to(device))
```

```
param_names = ['x', 'y', 'w', 'h']
```

Each number against the four it was never given

$z[0]$ rises with y , the top edge +0.69

$z[1]$ rises with h , the height +0.85

$z[2]$ falls as w , the width, grows -0.83

$z[3]$ rises with x , the left edge +0.78

Nobody told the encoder what a rectangle is. Four numbers had to be enough to rebuild the picture, and this is what they became.